

Pointfree CoEndoYoneda Lemma in Functional Categories Theory and Formalization in Lean 4

Luc Duponcheel Antigravity AI

July 23, 2026

Abstract

We present a variation of the CoYoneda Lemma for functional categories equipped with a monadic structure of its global elements, which we formally identify as *Functional Categories*. The central theoretical contribution, the Pointfree CoEndoYoneda Equivalence, establishes an isomorphism between, on the one hand, natural transformations mapping out of co-Yoneda endofunctors to some kind of global endofunctors and, on the other hand, some corresponding kind of global elements.

In addition to outlining the mathematical foundations, we detail the complete formalization of this result in the Lean 4 theorem prover. We discuss the technical challenges of working with functor compositions under strict definitional equality and demonstrate strategies for navigating dependent type theory to construct fully formal, pointfree proofs of higher-level category theory results.

1 Introduction

The *CoYoneda Lemma* serves as a cornerstone of modern category theory, asserting that for any category $\mathcal{C}$, object $X \in \mathcal{C}$, and functor $F : \mathcal{C} \rightarrow \mathbf{Set}$, natural transformations $\text{Nat}(\mathcal{C}(X, -), F)$ are isomorphic to the functor evaluations $F(X)$. This result fundamentally states that an object X is completely determined by the morphisms out of it.

In computational settings, we frequently encounter categories equipped with additional structure. Categories that have a functor mapping functions of sets to programs, their images being called pure programs. We define such a category as a *Functional Category*. In this paper, we explore a variation of the CoYoneda isomorphism extends in this setting. The variation replaces elements by global elements and is entirely pointfree.

We also provide a comprehensive discussion of how these concepts and theorems are formalized in the Lean 4 proof assistant using the `Mathlib` library.

2 Functional Categories

Let $\mathcal{C}$ be a locally small category. We denote the category of types (or sets) as **Type**. For any object $X \in \mathcal{C}$, the covariant Hom-functor, which we will call the co-Yoneda functor, is defined as:

$$CYF_X : \mathcal{C} \rightarrow \mathbf{Type}, \quad Y \mapsto \mathcal{C}(X, Y)$$

Definition 2.1. A *Functional Category* is a category $\mathcal{C}$ equipped with the following data:

1. A designated functor $\Phi : \mathbf{Type} \rightarrow \mathcal{C}$.
2. A constructed endofunctor $GF : \mathcal{C} \rightarrow \mathcal{C}$ defined by $GF(X) = \Phi(CYF_{\Phi(1)}(X))$.
3. A monadic structure on the globals of $\mathcal{C}$, provided by a unit natural transformation $\gamma\eta : Id_{\mathcal{C}} \rightarrow GF$ and a multiplication natural transformation $\gamma\mu : GF \circ GF \rightarrow GF$.

These structural components must satisfy the standard monad left unit law:

$$\gamma\eta_{GF(X)} \circ \gamma\mu_X = id_{GF(X)}$$

For now no other extra monad laws are needed in the formal Lean 4 proof. This is an interesting observation on its own.

This structure essentially allows $\mathcal{C}$ to reflect the category of types into itself, while treating GF as a monadic context. We define the *CoYoneda Endofunctor* $CYEF_X : \mathcal{C} \rightarrow \mathcal{C}$ for a fixed object X as the composition:

$$CYEF_X = \Phi \circ CYF_X$$

We use the following definition of *global element* as a morphisms from the canonical singleton $\Phi(1)$ to X .

$$G_\Phi(X) = \mathcal{C}(\Phi(1), X)$$

:

2.1 Formalization of Functional Categories in Lean 4

In Lean 4, a functional category is modeled as a type class extending the standard `LargeCategory` from `Mathlib`.

```

abbrev CYF {C : Type (u + 1)} [LargeCategory.{u} C] (X : C) : C  $\rightsquigarrow$  Type u :=
  coyoneda.obj (op X)

abbrev CYEF_def
  {C : Type (u + 1)} [LargeCategory.{u} C] ( $\Phi$  : Type u  $\rightsquigarrow$  C) (X : C) : C  $\rightsquigarrow$  C :=
  CYF X  $\ggg$   $\Phi$ 

abbrev GEF_def {C : Type (u + 1)} [LargeCategory.{u} C] ( $\Phi$  : Type u  $\rightsquigarrow$  C) : C  $\rightsquigarrow$  C :=
  CYEF_def  $\Phi$  ( $\Phi$ .obj PUnit)

class FunctionalCategory (C : Type (u + 1)) extends LargeCategory.{u} C where
   $\Phi$  : Type u  $\rightsquigarrow$  C

   $\gamma\mu$  : (GEF_def  $\Phi$   $\ggg$  GEF_def  $\Phi$ )  $\longrightarrow$  GEF_def  $\Phi$ 
   $\gamma\eta$  : 1  $\longrightarrow$  GEF_def  $\Phi$ 

   $\gamma$ _left_unit :
     $\forall$  (X : C),  $\gamma\eta$ .app ((GEF_def  $\Phi$ ).obj X)  $\gg$   $\gamma\mu$ .app X = id ((GEF_def  $\Phi$ ).obj X)

   $\varphi$  {X Y : Type u} (f : X  $\rightarrow$  Y) :  $\Phi$ .obj X  $\rightarrow$   $\Phi$ .obj Y :=  $\Phi$ .map (TypeCat.ofHom f)

   $\varphi$ _eq :
     $\forall$  {X Y : Type u} (f : X  $\rightarrow$  Y),  $\varphi$  f =  $\Phi$ .map (TypeCat.ofHom f) :=
    by intros; rfl

   $\gamma\eta$ _ $\varphi$  :
     $\forall$  (Z : Type u),  $\gamma\eta$ .app ( $\Phi$ .obj Z) =  $\varphi$  (fun z  $\Rightarrow$   $\varphi$  (fun _  $\Rightarrow$  z)) :=
    by intros; rfl

```

and global elements are modeled as

```

abbrev G
  {C : Type (u + 1)} [LargeCategory.{u} C] ( $\Phi$  : Type u  $\rightsquigarrow$  C) (X : C) : Type u :=
   $\Phi$ .obj PUnit  $\longrightarrow$  X

```

A canonical instance of the type class is **Type** itself, where Φ is the identity functor.

```

instance typesFunctionalCategory : FunctionalCategory (Type u) where
   $\Phi$  := 1 (Type u)
   $\gamma\eta$  := {
    app := fun X  $\Rightarrow$  (1 (Type u)).map ( $\lambda$  (f : fun x  $\Rightarrow$   $\lambda$  (f _  $\Rightarrow$  x)))
    naturality := fun X Y f  $\Rightarrow$  by ext; rfl
  }
   $\gamma\mu$  := {
    app := fun X  $\Rightarrow$  (1 (Type u)).map ( $\lambda$  (f : PUnit  $\longrightarrow$  PUnit  $\longrightarrow$  X)  $\Rightarrow$   $\lambda$  (f pu  $\Rightarrow$  f pu))
    naturality := fun X Y f  $\Rightarrow$  by ext; rfl
  }
   $\gamma$ _left_unit := fun _  $\Rightarrow$  by rfl

```

3 The CoEndoYoneda Equivalence

3.1 Auxiliary definitions and theorems

We construct a designated global element from a designated natural transformation as follows

```

def globalTransformationToGlobalGlobalValue {F : C  $\rightsquigarrow$  C} {X : C} ( $\tau$ X : CYEF X  $\longrightarrow$  F  $\ggg$ 
  GEF) :
  Global ((F  $\ggg$  GEF).obj X) :=  $\varphi$  (fun _  $\Rightarrow$  id X)  $\gg$   $\tau$ X.app X

```

We construct a designated (natural, not proved) transformation from a designated global element as follows

```

def globalGlobalValueToGlobalTransformation
  {F : C → C} {X : C} (ggfx : Global (GEF.obj (F.obj X))) : CYEF X → (F ≫ GEF) where
  app Y := φ (ggfx ≫ (F ≫ GEF).map .) ≫ γμ.app (F.obj Y)
  naturality _ _ h := by ...

```

We formulate (and prove, not shown) a left inverse theorem as follows

```

theorem left_inverse {F : C → C} {Z : C} (τX : CYEF Z → (F ≫ GEF)) :
  τX =
    globalGlobalValueToGlobalTransformation (φ (fun _ => id Z) ≫ τX.app Z) := by ...

```

We formulate (and prove, not shown) a right inverse theorem as follows

```

theorem φ_γμ {X : C} (ggfx : Global ((GEF.def Φ).obj X)) :
  φ (fun _ => ggfx) ≫ γμ.app X =
    ggfx := by ...

```

3.2 CoEndoYoneda Equivalence

Utilizing the auxiliary definitions and theorems above we can now formulate (and prove, not shown) the main equivalence.

```

def coEndoYonedaEquiv {F : C → C} {X : C} :
  (CYEF X → (F ≫ GEF)) ≃ Global ((F ≫ GEF).obj X) where
  toFun      := globalTransformationToGlobalGlobalValue
  invFun     := globalGlobalValueToGlobalTransformation
  left_inv   := fun τX => (left_inverse τX).symm
  right_inv  := fun τX => (right_inverse τX).symm

```

This equivalence provides a completely pointfree and logically verified method for evaluating and constructing natural transformations in functional categories.

4 Proof Techniques and Challenges in Lean 4

Formalizing abstract category theory in dependent type theory (DTT) introduces unique difficulties, particularly concerning definitional equality.

4.1 Strict Functor Composition

In standard mathematics, functor composition is strictly associative. In Lean 4, however, these are represented as distinct abstract syntax trees. They are *isomorphic* via `Category.assoc`, but they are not *definitionally equal*.

When writing the proof of `left_inverse`, this limitation prevents standard term-rewriting tactics like `simp` from effortlessly reducing deeply nested compositions.

4.2 Constructing Equality Chains

To circumvent the strict type-checker without resorting to manual casts (such as `eqToHom`), the proof for `left_inverse` abandons the monolithic `simp` approach in favor of explicit equality chains acting on the natural transformation components.

By utilizing `Eq.trans` and `congr_arg`, we explicitly guide the type checker through the unit laws and naturality squares:

```

have h1 :
  τX.app Y = τX.app Y >> id ((GEF_def Φ).obj (F.obj Y)) :=
  (Category.comp_id (τX.app Y)).symm
have h2 :
  id ((GEF_def Φ).obj (F.obj Y)) =
  γη.app ((GEF_def Φ).obj (F.obj Y)) >> γμ.app (F.obj Y) :=
  (γ_left_unit (F.obj Y)).symm
have h_left_unit :
  τX.app Y =
  τX.app Y >> (γη.app ((GEF_def Φ).obj (F.obj Y)) >> γμ.app (F.obj Y)) :=
  Eq.trans h1 (congr_arg (τX.app Y >> .) h2)

```

This localized approach bypasses the rigid typing of the $\longrightarrow$ category morphisms by acting directly on the equality type `=` in the universe of types. This technique is highly robust and avoids the infamous "motive is not type correct" errors common in Lean category theory formalization.

5 Conclusion

The Pointfree CoEndoYoneda Lemma is a variation of the classical Pointful CoYoneda Lemma in a functional category setting. By successfully formalizing the lemma in Lean 4, we have not only verified its absolute logical correctness but also established a concrete methodology for handling strict composition equality in dependent type theory. The approach of combining explicit structural definitions with targeted, step-by-step equality chaining provides a reliable method for formalizing advanced category theory in modern proof assistants.